CausalAtlas

Working with Large Language Models

Auditable mechanistic reasoning for biomedical discovery

Technical Project Report

Dmitrii Zakharenko

Instructors: Suhail Yazijy and Katharina Matulla

29 July 2026

Repository: https://github.com/dmitriizakharenko/causalatlas

1. Executive summary

CausalAtlas is a vertical biomedical-research agent for turning a disease–target question into an auditable mechanistic argument. Its output is not a free-form paragraph: it is a provenance-backed evidence graph, a documented novelty classification, a falsifiable hypothesis and an experiment design.

The architecture combines language-model agents with deterministic tool-backed agents. LLMs handle interpretation-heavy tasks such as literature reasoning, mechanistic extraction, novelty classification, hypothesis generation and peer review. Deterministic code handles persistence, event translation, graph merging, loop detection, topology metrics, contradiction/gap scans and API behavior. This division improves reproducibility and prevents safety-critical structure from depending on model fluency alone.

- Agentic architecture: sequential agents use tools, artifacts and checkpoints to solve a multi-stage research problem.
- Context engineering: the root AGENTS.md, per-agent contracts and reusable SKILL.md procedures provide structured context and durable constraints.
- Autonomy slider: autocomplete, supervised and let_it_rip modes control where human approval is required.
- Evaluation flywheel: historical failure cases are scored in an eval dashboard, with an optional independent live judge.
- Safety: provenance, contradiction preservation, novelty gating, explicit uncertainty and falsification criteria constrain unsupported claims.

2. Problem and motivation

Biomedical mechanism discovery has a specific failure mode: a model can produce a plausible causal story that is already published, weakly sourced or impossible to falsify. A stronger single prompt does not solve this architectural problem because retrieval, validation, novelty checking and experiment design have different incentives and failure modes.

CausalAtlas separates the workflow into explicit stages:

PIPELINE

Disease + target → canonical baseline → literature → verification → quality → extraction → graph → novelty gate → hypothesis → peer review → experiment.

The knowledge graph is deliberately treated as a graph of published evidence, not as biological truth. Conflicting edges remain visible with separate provenance.

3. System architecture

3.1 Runtime layers

Layer	Responsibility	Location
Context	Project constraints, agent contracts and reusable procedures	AGENTS.md, agents/, skills/
Execution	Provider adapters, orchestration, JSONL translation, persistence, API and SSE	backend/app/
Scientific state	Run artifacts, checkpoints and cumulative disease graphs	data/sessions/, data/graphs/
User experience	Launch, runs, graph explorer, evidence, eval, replay and presentation	frontend/src/

3.2 Agent model

In this report, “agent” means a goal-bounded component with an explicit contract, input/output artifacts and a place in the decision process. It does not imply that every stage must call an LLM.

Stage	Type	Role
01–05	LLM/tool-assisted	Canonical baseline, literature retrieval, publication verification, quality interpretation and mechanistic extraction
06–09	Deterministic/tool-backed	Graph merge, loop discovery, topology analysis and contradiction/gap detection
10–13	LLM/tool-assisted with gates	Novelty verification, hypothesis generation, peer review and experiment design
00	Orchestration	Sequencing, autonomy pauses, persistence and resume behavior
14	Independent eval	Blind live judge for a completed hypothesis; never part of hypothesis generation

The Claude provider uses the native CLI orchestration path. The Codex provider uses sequential codex exec --json calls for language-heavy stages and deterministic Python for graph stages. Both providers are normalized into the same UI event model.

4. Context engineering

Context is separated into two layers:

- AGENTS.md defines who is responsible for a task, required inputs and outputs, hard constraints, failure rules and success criteria.

- SKILL.md defines reusable procedures such as PubMed retrieval, canonical database lookup, novelty verification, contradiction detection and graph export.

The runtime injects relevant contracts, skills, upstream artifacts, budgets and checkpoint paths into each stage. This makes the procedure reviewable and editable without burying the scientific method inside Python code.

- Never invent papers, PMIDs, identifiers or experimental results.
- Retain parallel edges when evidence conflicts.
- Keep canonical database provenance distinct from PMID provenance.
- Treat zero-hit searches as insufficient evidence by themselves.
- Route A/B/C novelty outcomes away from hypothesis generation.
- Persist failed and paused states rather than fabricating downstream output.

5. Autonomy slider and human control

Mode	Human checkpoint behavior
autocomplete	Pause after every agent
supervised	Run evidence construction, then pause at novelty and experiment checkpoints
let_it_rip	Run without approval pauses while retaining the full trace

Approval, rejection and edit decisions are persisted as interventions. Autonomy changes execution policy; it does not weaken novelty or safety rules.

6. Evidence and novelty safety

The novelty gate is the most safety-critical component. It has two distinct checks:

- Structural originality: reject a candidate as RESTATED when it substantially repeats a single source paper’s conclusion.
- Independent external search: search the specific causal chain, record queries and results, and classify the candidate as A–E.

Class	Routing
A	Established consensus: fold into the evidence graph.
B	Previously published: fold into the graph.
C	Conflicting literature: route to contradiction handling.
D	Partially established: eligible for hypothesis generation.
E	Potentially novel: eligible for hypothesis generation, subject to review.

GATE RULE

Only D/E candidates can proceed. They are eligibility classes, not guarantees of novelty or truth.

7. Evaluation flywheel

The project includes a deterministic historical backfill for Sessions 001–003. These sessions contain the founding failure cases: a previously accepted candidate that was a single-paper restatement and another mechanism that had already been established in the literature.

- Total scored sessions and covered historical sessions.
- Outcome counts for false positives, correct rejections and pending validation.
- Historical false-positive catch rate.
- Future live-judge agreement rate.

The verified offline backfill produced five historical eval records and a retroactive catch rate of 1.0 for the documented historical gate failures. The optional agent14_eval_judge is separated from the pipeline and runs only when an auditor requests it for a completed hypothesis.

8. Prototype interface and demo

- Launch & Runs: disease/target input and autonomy selection.
- Graph Explorer: graph selection, node search, provenance and edge inspection.
- Agent Architecture: contracts, skills and deterministic stages.
- Evidence Dashboard: evidence, novelty, hypotheses and experiment artifacts.
- Eval Dashboard: historical and future judge results.
- Recorded Demo: read-only guided replay from an embedded completed-run snapshot.
- Presentation: browser-based narrative walkthrough.

Recommended manual routes after startup:

- http://127.0.0.1:5173/?offline=1
- http://127.0.0.1:5173/demo.html#/demo
- http://127.0.0.1:5173/presentation
- http://127.0.0.1:8000/docs

The recorded replay is intentionally read-only and does not require credentials, PubMed access or LLM quota.

9. Reproducibility

The exact procedure is documented in docs/REPRODUCIBILITY.md. The deterministic offline verification path is:

```
cd backend
./.venv/bin/python -m pytest -q
```

```
cd ../frontend
npm ci
npm run lint
npm run build
```

- Backend: 161 passed, 6 live tests deselected.
- Frontend: lint passed.
- Frontend: TypeScript build passed.
- Frontend: Vite production build passed.
- API health: status ok, Codex provider, 13 pipeline agents.
- Recorded replay endpoint: completed read-only replay available.

Live LLM runs are intentionally opt-in. They require a locally authenticated Claude or Codex CLI and may consume external quota. No browser credential or API key is required by the offline demo.

10. Security and limitations

The repository tracks .env.example, not local .env files. Credentials remain in the CLI's user-level configuration and are never placed in frontend VITE_* variables.

- Live literature and LLM quality depends on external services and searchable coverage.
- Novelty classification is a documented search-based judgment, not proof of global absence.
- Deterministic agents improve repeatability but cannot replace scientific expert review.
- The project is research software, not medical advice or a clinical decision system.

11. Use of generative AI and declaration of authorship

Generative AI tools, specifically Claude Code and OpenAI Codex, were used in accordance with the learning objectives of this course to support coding, debugging, documentation, agent orchestration and testing. Their use was limited to the extent permitted by the course requirements and remained under the author's direction, critical review and responsibility. Project decisions, output evaluation and implementation verification were performed by the author, who retains responsibility for the final work.

This assignment is the sole work and composition of Dmitrii Zakharenko. All sources and learning aids used in its preparation, including the generative AI tools stated above, have been acknowledged. Precise and detailed references to the work of others have been provided, and the author takes full responsibility for the submitted assignment.